

AGENT AS POLICY FOR ROBOTIC MANIPULATION

Mengzhao Jia^{1,*} Yang Lin^{2,*} Xixin Zhang^{3,*} Zhihan Zhang¹
 Xiaobai Liu³ Meng Jiang¹

¹University of Notre Dame ²University of California San Diego ³San Diego State University

*Core contributors

¹mjia2@nd.edu

ABSTRACT

We demonstrate that a general-purpose agent can directly drive a physical robot throughout task execution without any task-specific or environment-specific training. We introduce **Agent as Policy** (AGP), which places task planning and execution under the agent’s control. Given a task and a robot interface, the agent interprets visual evidence, writes executable programs, issues motion commands, and revises its actions in response to physical outcomes. This brings the agent’s reasoning and programming capabilities into continuous interaction with the physical world. We study AGP across multiple real-world manipulation tasks spanning precision manipulation, dynamic motions, and deformable objects. These include assembly from human videos, block construction from goal images, dice flipping, targeted throwing, and bimanual towel folding. Across assembly, block construction, and dice flipping, AGP succeeds in at least eight of ten trials for each evaluated task configuration. We further study efficiency through task experience accumulation and find that reusing saved procedures and programs shortens execution time across repeated trials. These findings support a path for general-purpose agents to act as robot policies, extending their autonomy to physical manipulation through runtime reasoning, programming, and interaction.

1 INTRODUCTION

Multimodal large language models (MLLMs) demonstrate strong general capabilities (Li et al., 2026b). Agents built on these models tackle digital tasks such as software development and computer use (Yang et al., 2024; Wang et al., 2024; Xie et al., 2024). Recent work extends these agents to robot manipulation, bringing their task execution capabilities from digital environments into the physical world (Li et al., 2026a; Zhang et al., 2026; Galanti et al., 2026). These agentic robot systems broadly follow two approaches. The first approach uses agents to generate executable programs in advance, expressed as code or computation graphs that the robot runs during task execution (Liang et al., 2022; Chen et al., 2026). The second approach uses agents as orchestrators of existing learned policies, selecting and sequencing these policies according to task goals and execution feedback (Li et al., 2026a; Zhang et al., 2026; Galanti et al., 2026). Both approaches, however, have limitations. Generated programs must express runtime decisions through explicit rules, making it difficult to anticipate how uncertain observations and unexpected outcomes should affect execution. Policy orchestration gives agents control over which policy runs next, while motion generation remains limited by what the selected policies can perform.

Yet physical tasks often require the flexibility to adapt both how evidence is interpreted and how actions are generated. After a failed insertion, for example, the robot may need a closer view, a revised alignment estimate, or a different approach direction. Choosing and carrying out the appropriate response requires reasoning jointly about the current scene, the attempted motion, and its outcome. To this end, we introduce **Agent as Policy** (AGP), which uses the agent itself as the policy throughout task execution. We grant the agent full autonomy over runtime task decisions: it decides what to observe, how to interpret evidence, and which motion to request, and can author and revise local programs as needed. Each result informs the next decision, including whether to gather more evidence or change the execution strategy. This gives the agent control over perception, motion generation, and recovery at the level where new physical evidence becomes available.

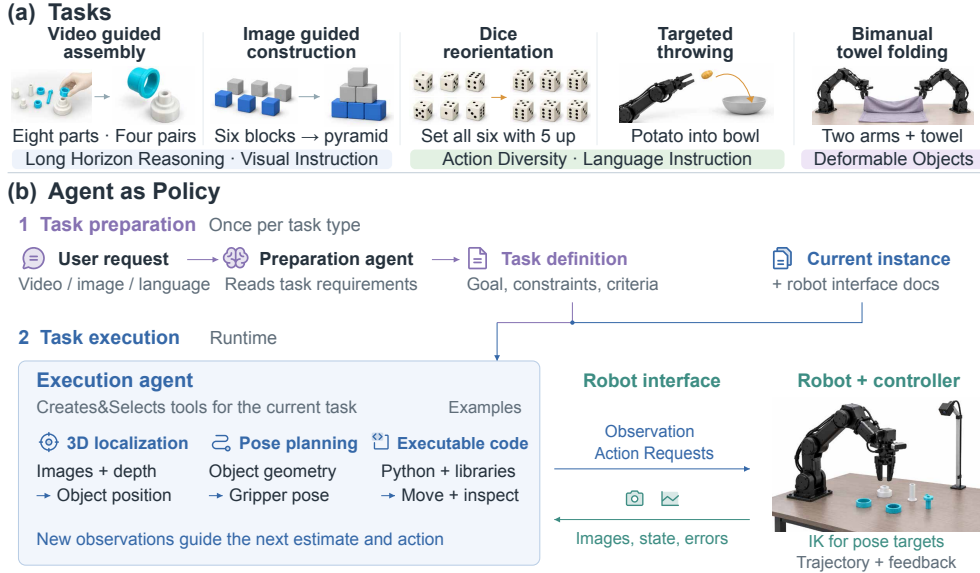


Figure 1: Overview of AGP. (a) Evaluated tasks include video guided assembly, image guided construction, dice flipping, targeted throwing, and bimanual towel folding. (b) A preparation agent creates a reusable task definition. At runtime, the execution agent writes and selects programs to interpret observations, compute motion targets, and request robot actions through the robot interface. After each action, the agent uses motion feedback and new observations to refine its estimates and adjust subsequent actions.

We implement AGP by connecting a general purpose coding agent to the robot through a bridge that exposes calibrated observations, geometric queries, and motion commands (Figure 1). The coding agent provides the programming tools and persistent workspace used to create scripts, inspect data, and retain execution evidence. Through these tools, it can compute object geometry, evaluate candidate motions, and revise action sequences using feedback from the robot. The bridge grounds these computations in camera calibration and robot coordinates, validates requested motion, and returns measured state and execution reports. It also supports coordinated joint and gripper motion for actions that depend on timing. Motion execution and monitoring run independently of agent inference, allowing the robot to execute a submitted motion while the agent reasons between tool calls. Adaptation occurs through updated measurements, programs, and decisions within the session, with model parameters fixed throughout execution.

We evaluate AGP directly on real robots through assembly from human videos, block construction from goal images, dice flipping, targeted throwing, and bimanual towel folding. In the main evaluation, each trial starts in a fresh session with fixed model weights and no prior task execution experience or simulation rehearsal. With ten trials per configuration, AGP succeeds in 8 of 10 trials on assembly, 29 of 30 trials across three block construction configurations, and 10 of 10 trials on dice flipping. We further study experience accumulation and transfer by having the agent record procedures, measurements, and corrections in files for reuse in subsequent executions. Our experiments show that accumulating these records improves execution efficiency. Transferring these records from a stronger agent improves the weaker agent’s task success rate and reduces its mean execution time and inference cost on successful trials.

This work provides evidence that general purpose foundation MLLMs can serve as policies for direct robot arm control through a robot interface. Their visual perception and reasoning capabilities support interpreting task goals, generating motions, and adapting to physical feedback during zero shot manipulation. The substantial execution time and inference cost remain barriers to practical deployment. We hope this study provides insights for future research on using the powerful capabilities of foundation MLLMs for more capable and efficient robot manipulation.

2 RELATED WORK

2.1 AGENTIC ROBOT SYSTEMS

Agentic robot systems use language models for reasoning, tool use, and adaptation through execution feedback, with two main roles for the agent. **Program synthesis and refinement.** One line generates programs that map observations to actions through perception and control interfaces, with validation supporting execution in simulation and on physical robots (Singh et al., 2022; Liang et al., 2022; Chen et al., 2024; Mu et al., 2024). Recent work uses execution evidence to iteratively refine programs or computation graphs and retain reusable solutions (Lu et al., 2026). AGP keeps a runtime agent in the decision loop during deployment, allowing it to choose new approaches, write new programs, and revise workflows based on observations and execution feedback. **Runtime policy orchestration.** A second line selects and sequences learned skills according to task goals, affordances, and environment feedback (Ahn et al., 2022; Huang et al., 2022). Recent systems extend this approach to vision language action (VLA) policies with progress monitoring and recovery (Li et al., 2026a; Zhang et al., 2026; Galanti et al., 2026). These systems separate skill selection from motion generation within the invoked policy or primitive. AGP uses the runtime agent itself as the policy without a separately trained action policy, enabling zero shot task execution in previously unseen environments.

2.2 ROBOT MANIPULATION

Classical robot manipulation combines geometric modeling, task and motion planning, and feedback control (Garrett et al., 2021). Learned visuomotor policies predict actions from robot trajectory data, with diffusion models capturing action sequences (Chi et al., 2023). Vision language action models extend this approach to diverse tasks through image and language conditioning (Brohan et al., 2023; Kim et al., 2024; Black et al., 2024). Video and world action models further couple actions with future visual states to inform action generation and evaluation (Li et al., 2025; Ye et al., 2026). AGP introduces an alternative to policies trained specifically for action generation by using a general purpose foundation multimodal large language model (MLLM) directly as the robot policy.

2.3 AGENTIC WORKFLOWS BEYOND ROBOTICS

Language agents combine reasoning, tool use, and reflection on interaction feedback (Yao et al., 2022; Schick et al., 2023; Shinn et al., 2023). Coding agents apply these mechanisms to editing and testing software (Yang et al., 2024; Wang et al., 2024), while computer use agents act through browser and desktop interfaces, adapting to interface changes (Zhou et al., 2023; Xie et al., 2024). Multiagent frameworks distribute workflows among specialized roles that communicate and share intermediate results (Wu et al., 2023; Hong et al., 2023). These workflows establish mechanisms for tool use, memory, feedback, and coordination. AGP studies how they shape the robot policy boundary, with agents governing physical task execution through perception, geometry, planning, and control interfaces.

3 A GENERAL PURPOSE AGENT AS THE ROBOT POLICY

3.1 PROBLEM FORMULATION

Task objective. We consider physical manipulation tasks specified through video instructions, image instructions, language instructions, or a combination of these. A robot operates in a persistent scene, and the objective is to bring that scene into a state satisfying the task criteria within fixed budgets for elapsed time and observation and action requests.

Task preparation. When a task type is first introduced to the system, a separate coding agent prepares a reusable task definition from the user’s request. This agent runs only during task preparation. A different agent, the execution agent, carries out the task at runtime. This definition specifies the goal, reference materials, constraints, allowed variations, and completion criteria, along with interface access rules, budgets, and reporting requirements. For subsequent instances of an existing task type, a launch program starts the execution agent directly with the saved definition and the current instance details. The execution agent reads the supplied materials and carries out the requested instance.

Agent as the robot policy. We use a general purpose agent as the robot policy. The agent receives a task specification g , including the prompt and reference materials, and a documented robot interface I . It maintains a local workspace W containing scripts, notes, and observations collected during execution. The agent interprets the task, plans actions, and interacts with the robot through tools. We describe its tool selection process as

$$u_k \sim \pi_\theta(\cdot \mid g, I, H_k, W_k), \quad (1)$$

where u_k is the tool call selected at step k , H_k contains the conversation and tool results available at that step, and W_k contains the current workspace files. A tool call may acquire information or execute a program that submits action targets to the robot controller. The model parameters θ remain fixed throughout execution.

3.2 AGENT-ROBOT BRIDGE

The robot interface I supports information exchange in two directions. The robot provides camera observations and proprioceptive state to the agent. The agent sends requests for observations or actions through this interface. Together, these exchanges allow the agent to observe the scene, direct physical actions, and assess the resulting state.

Robot to agent. The interface provides two sources of information. Cameras supply overhead images for a view of the scene and wrist images with aligned depth for closer inspection and spatial measurement. Camera calibration and poses relate these observations to a shared robot coordinate frame. The robot’s state interface supplies joint positions from motor feedback and the corresponding end effector pose computed through forward kinematics. This proprioceptive state describes the robot’s current configuration and supports comparison with commanded targets. After a motion, the interface returns the available updated state and target errors computed from this state. Observations are saved in the agent’s workspace for visual inspection and programmatic processing.

Agent to robot. The agent sends requests through the interface. These include observation requests, which ask the robot system to capture images or report its state, and action requests, which specify arm movements or gripper opening and closing. For arm movement, the agent chooses between specifying target joint angles and specifying a target end effector position and orientation. The latter describes where to place the gripper and how to orient it. Gripper requests separately specify how far the fingers should open or close. To carry out these requests, the robot uses a controller to turn the agent’s targets into commands that drive its joints and gripper. For end effector targets, it uses Mink (Zakka, 2026) for inverse kinematics to compute joint targets. It then generates joint trajectories subject to workspace and motion limits and uses motor feedback to track them. Between requests, the controller maintains the arm’s posture while the agent reasons about its next step. Motion commands execute in order for each arm, and commands to different arms can proceed concurrently.

3.3 RUNTIME PROGRAMMING AND EXECUTION

Runtime programming. The agent writes programs during execution to interpret observations, compute action targets, and submit them through the robot interface. It can inspect reference videos and camera images and process saved data with Python and available libraries. For example, it can estimate an object’s position from camera observations and depth measurements, compute a target gripper pose, and issue the corresponding robot commands. Appendix E analyzes recorded programs for geometric estimation, visual processing, and robot call composition.

Feedback and adaptation. After executing an action, the agent uses motion feedback and new observations to revise its estimates and subsequent actions. For example, an unsuccessful insertion can prompt a new alignment estimate and a modified approach. The agent determines grasp poses, action sequences, observation timing, and recovery during this cycle. It continues until it reports completion or ends the attempt within the task budget. Physical success is assessed against the task criteria using the resulting scene and recorded evidence.

Execution efficiency. To reduce tool overhead, we instruct the agent to wait longer for completed action results and to combine consecutive operations, such as moving the arm and then capturing and displaying an image, in one tool call. These instructions are enabled according to the experimental condition.

4 EXPERIMENTS

We evaluate AGP on real-world robot tasks spanning long horizon reasoning, action diversity, and object diversity under video, image, and language instructions. We assess success rate, completion time, token usage, and inference cost, then examine whether accumulated experience improves execution efficiency.

4.1 EXPERIMENTAL SETUP

4.1.1 ROBOT PLATFORM AND AGENT CONFIGURATION

Robot Platform. We use an I2RT YAM (I2RT) arm with six revolute joints and a parallel gripper in a tabletop workspace. AGP provides the robot interface described in Section 3.2. Bimanual towel folding uses two arms with a common coordinate frame and coordination selected by the agent. The targeted throwing experiment uses an interface with timed motion programs, documented in Appendix A.1.

Cameras. A wrist mounted Intel RealSense D405 provides aligned RGB and depth at 640×360 pixels. A fixed overhead Logitech BRIO provides rectified RGB at 1920×1080 pixels. Both cameras capture at 30 frames per second, and the agent requests observations on demand.

Model Configuration. GPT-6 Astra (OpenAI, b) at high thinking effort is the default across all four task groups. A separate study varies the model and thinking effort on a two pair assembly task. Model weights remain fixed. Appendix A gives platform, agent, and model configuration details.

4.1.2 TASKS AND EVALUATION

Capability Driven Task Design. We find that conventional pick and place tasks, such as placing an object into a bowl, pose little challenge for AGP, motivating more demanding tasks to evaluate its capabilities. We use four task groups. Assembly of eight parts into four pairs from a human video and block construction from goal images test *long horizon reasoning* under video and image instructions, respectively. Both require interpreting visual relationships and ordering operations while preserving earlier progress. The assembly parts are adapted from the AutoMate dataset (Tang et al., 2024). Block targets comprise a pyramid, two towers, and a six block tower. Dice flipping requires reorienting six randomly placed dice so that all six show the requested number on their upward faces. Dice flipping and targeted throwing follow language instructions and test *action diversity* through grasp dependent reorientation and timed release during a circular swing. Bimanual towel folding follows video instructions and tests *object diversity* through coordinated manipulation of deformable material. Appendix B.1 gives the design rationale, task definitions, and success criteria.

Trial Protocol. Trials use predefined initial scenes and fixed budgets for elapsed time and observation and action requests. Model and experience comparisons use matched scenes and counterbalanced execution order. Recovery is permitted within these budgets. A reset or human intervention ends the autonomous trial. Physical outcomes and execution video determine success using predefined criteria. Appendix B.2 gives the detailed protocol.

Metrics. Table 1 reports AGP success rates and resource use across tasks. Time is the mean elapsed minutes from task delivery to final physical verification over successful trials, with the observed minimum and maximum shown below the mean. Tokens reports mean total input and output usage over successful trials in millions, with the observed minimum and maximum below, including reasoning tokens and retries. Block construction and towel folding report success rate, completion time, token usage, and model inference cost separately for each equally sampled configuration. Appendix B.3 defines the metrics and resource accounting.

4.2 EXPERIMENTAL RESULTS

4.2.1 MAIN RESULTS

Table 1 summarizes AGP success rates, completion times, token usage, and inference costs across the manipulation tasks. The results yield three main observations. **Strong zero shot performance.** AGP achieves success rates of at least 80% in seven of eight task configurations, with 100% observed

Table 1: **Main results across manipulation tasks.** Success is reported as the number of successful trials over the total number of trials (n/N). Time, tokens, and cost summarize successful trials, with means above [min, max]. Cost uses Standard API rates and accounts for cache hits through cached input pricing. † Throwing time includes subsequent motion trajectory analysis and verification. The throw itself is completed an average of 13.9 minutes after task delivery. Tags denote long horizon reasoning (LHR), action diversity (AD), object diversity (OD), video instruction (Vid), image instruction (Img), and language instruction (Lan).

Task	Capability	Instruction	Success (n/N)	Time ↓ (min)	Tokens ↓ (million)	Cost ↓ (USD)
Four pair assembly	LHR	Vid	8/10	37.2 [16.0, 59.1]	13.02 [3.96, 27.12]	16.62 [5.55, 31.50]
Block construction	LHR	Img				
<i>Pyramid</i>			10/10	21.6 [14.9, 32.2]	9.24 [6.27, 12.94]	11.69 [8.01, 15.28]
<i>Two towers</i>			10/10	20.6 [14.8, 29.2]	7.94 [4.94, 14.39]	9.92 [6.27, 16.85]
<i>Six block tower</i>			9/10	28.2 [11.6, 55.0]	11.57 [4.50, 19.85]	14.93 [6.50, 26.78]
Dice flipping	AD	Lan	10/10	37.9 [22.6, 56.0]	17.08 [10.16, 25.00]	21.07 [12.57, 31.20]
Targeted throwing	AD	Lan				
<i>Potato</i>			2/2	22.8 [†] [22.2, 23.3]	6.26 [6.25, 6.26]	8.94 [8.71, 9.18]
Towel folding	OD	Vid				
<i>Sequential</i>			5/5	50.8 [33.8, 63.3]	18.12 [10.13, 25.26]	24.14 [14.16, 30.90]
<i>Simultaneous</i>			3/5	21.4 [15.7, 31.2]	5.06 [4.27, 5.68]	7.46 [6.16, 8.67]

success in five. These results support its effectiveness for zero shot manipulation across the evaluated tasks. **Task complexity and execution overhead.** Execution time and inference cost generally increase with task complexity, as illustrated by the six block tower relative to the pyramid and two towers. Execution overhead also includes reviewing actions and verifying outcomes. In potato throwing, the throw is completed after 13.9 minutes on average, while subsequent trajectory analysis and verification account for approximately 40% of the reported 22.8 minutes. Appendix D.1 provides a detailed breakdown of execution time across the main tasks. **Challenges in deformable object manipulation.** Deformable object manipulation remains challenging. Sequential towel folding succeeds in all five trials but requires 50.8 minutes and USD 24.14 on average, among the highest expenditures in the task suite. Simultaneous folding, whose demonstration brings both short ends inward together, has the lowest observed success rate at 3/5, highlighting the challenge of coordinating dynamic manipulation of deformable material.

4.2.2 AGENT AND THINKING EFFORT COMPARISON

Table 2 compares agent and model configurations on two pair assembly from a human demonstration. The task pairs a hexagonal ring with a short post and a circular sleeve with a stepped cylinder. Within Codex (OpenAI, a), we evaluate GPT-6 Astra at low, medium, and high thinking effort and GPT-5.6 Sol, Terra, and Luna (OpenAI, c) at high thinking effort. The comparison also includes Claude Code (Anthropic, a) with Claude Opus 5 (Anthropic, 2026) and Claude Fable 5.1, both at high effort. Task references, robot functions, and trial budgets are shared across configurations.

AGP supports successful assembly across both Codex and Claude Code, while the performance differences among models within each agent show that model choice remains important. Astra maintains consistent success across thinking effort levels, suggesting limited benefit from additional thinking effort on this task. Sol offers lower inference cost with longer execution time than Astra.

Table 2: **Agent and thinking effort comparison on two pair assembly.** GPT-6 Astra is reported at low, medium, and high effort as separate rows. Other models use high effort. Aggregation follows Table 1. Success is reported as the number of successful trials over the total number of trials (n/N). Model versions, native effort settings, and agent versions are recorded before collection.

Model	Success (n/N)	Time ↓ (min)	Tokens ↓ (million)	Cost ↓ (USD)
<i>Codex</i>				
GPT-6 Astra				
<i>Low effort</i>	5/5	9.9 [6.0, 19.0]	3.69 [1.44, 7.72]	4.79 [2.15, 9.56]
<i>Medium effort</i>	5/5	9.2 [6.3, 15.4]	3.03 [1.84, 5.80]	4.09 [2.76, 7.48]
<i>High effort</i>	5/5	9.2 [7.9, 10.6]	3.30 [2.58, 4.02]	4.47 [3.72, 5.19]
GPT-5.6 Sol	5/5	14.2 [9.1, 19.3]	7.13 [4.12, 9.72]	3.94 [2.40, 5.30]
GPT-5.6 Terra	1/5	19.0 [19.0, 19.0]	6.28 [6.28, 6.28]	1.98 [1.98, 1.98]
GPT-5.6 Luna	0/5	—	—	—
<i>Claude Code</i>				
Claude Opus 5	5/5	22.2 [14.4, 31.9]	12.40 [9.05, 16.73]	9.75 [6.94, 13.03]
Claude Fable 5.1	3/5	27.7 [18.3, 39.8]	9.27 [6.87, 11.10]	11.30 [8.33, 14.36]

Within Claude Code, Fable achieves lower success than Opus and uses fewer tokens on successful trials, yet incurs greater time and cost.

4.3 IMPROVING EFFICIENCY THROUGH EXPERIENCE

Table 1 demonstrates strong task success rates for AGP, but execution time and token usage remain substantial. The resulting latency and inference cost can pose challenges for practical deployment, particularly when tasks must be performed repeatedly. We therefore investigate two ways to reduce these costs through task experience. The first lets an agent accumulate experience over repeated executions of the same task, potentially becoming faster as it refines and reuses its procedures. The second lets a relatively stronger but more costly agent first attempt an unfamiliar task and establish a successful procedure through its own execution. It then passes the resulting experience to a relatively weaker but more economical agent for subsequent execution.

To limit evaluation costs, we select two pairings from the four pair assembly task in Section 4.1.2 for both experience accumulation and transfer. The resulting task requires fitting a hexagonal ring onto a narrow post and a circular collar onto a stepped cylinder.

4.3.1 ACCUMULATING REUSABLE TASK EXPERIENCE

People build experience through repeated task execution, retaining successful procedures and lessons from mistakes that can make later attempts more efficient. Persistent memory carries execution feedback across trials (Shinn et al., 2023). Motivated by this process, we investigate whether an agent can improve its efficiency by accumulating and reusing task experience.

We perform five executions of the two pair assembly task using GPT-6 Astra at high thinking effort. As a complementary task, we also evaluate five cycles of disassembly and reassembly of a ridged ring and its base. Each task uses a fixed robot and execution configuration, with model weights held constant. The first execution begins with empty experience. After each execution, the agent saves measurements, procedures, corrections, and scripts as files. Each subsequent execution starts with a fresh agent context that reads these files, updates object poses from current observations, and extends the saved experience. We measure task execution time and experience writing time separately to account for the overhead of retaining experience.

Figure 2 shows shorter task execution times with accumulated experience on both tasks. Ring disassembly and reassembly becomes faster after the first cycle and remains near that lower task time. Two pair assembly becomes progressively faster, with task time decreasing by 29.3% from the first to the fifth execution, while experience writing remains relatively stable. Analysis of the agent execution traces shows that subsequent assembly trials reuse geometric estimates while updating object positions from current observations. They also retain validated grasp and insertion parameters, incorporate corrections from earlier execution feedback, and reuse measurement and observation processing scripts. The timing breakdown in Appendix D.2 separates tool execution from the remaining response latency, which jointly includes reasoning and programming. For two pair assembly, this response latency decreases by 47.0% from the first to the fifth execution, while tool execution increases by 9.3%. The reduction is consistent with the observed reuse of programs and procedures. These dependent trials support an association between experience reuse and improved execution efficiency.

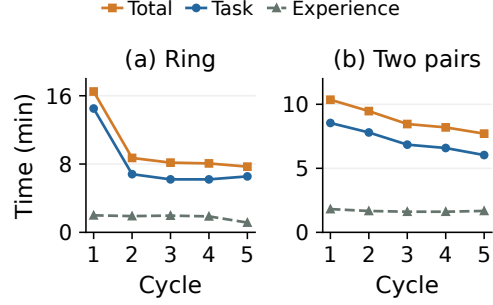


Figure 2: **Experience reuse across two tasks.** Total runtime is the sum of task execution and experience writing, as defined in Appendix D.2.

4.3.2 STRONG-TO-WEAK EXPERIENCE TRANSFER

Our second study examines whether a relatively stronger but more costly agent can first complete a task, summarize its experience, and pass the successful procedure to a relatively weaker but more economical agent. This could spread the cost of initial exploration across subsequent executions, allowing the more economical agent to handle repeated tasks at lower inference cost. We use GPT-6 Astra to collect experience and GPT-5.6 Terra to reuse it on the two pair assembly task in Table 2.

Starting with empty task experience, Astra saves geometry, procedures, corrections, and scripts as files over five executions. We compare five Terra trials with empty experience against five with read access to these frozen files, measuring success, completion time, token usage, and inference cost. All Terra trials use fresh agent contexts, fixed model weights, and high thinking effort. Task references, robot interface, and limits on elapsed time and observation and action requests match the model comparison in Section 4.2.2. Success requires both demonstrated pairings to remain engaged and stable after release.

Success increases from 1/5 with empty experience to 4/5 with Astra experience. Among successful trials, mean completion time and token usage decrease by 34.3% and 11.4%, respectively (Figure 3). Mean inference cost also decreases, excluding Astra experience collection. These results demonstrate that transferring experience from a stronger agent to a weaker agent improves task success and reduces the time and inference cost of successful executions on this task. Appendix D.3 documents the Astra to Terra transfer setup, trial outcomes, and calculation of completion time, token usage, and inference cost.

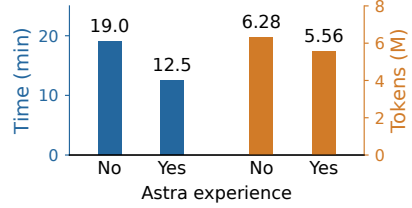


Figure 3: **Transferring Astra experience to Terra.** Mean time and token usage over successful trials. Success increases from 1/5 to 4/5.

5 CONCLUSION

We introduce AGP, which uses a general purpose agent as the robot policy. Through runtime programming and a robot interface, the agent interprets observations, generates motion requests, and revises its actions using physical feedback. Real robot experiments evaluate long horizon reasoning, action diversity, and object diversity under video, image, and language instructions. The results support AGP’s ability to carry out diverse manipulation tasks. Our experiments also show that task

experience recorded in reusable files improves execution efficiency as it accumulates. Transferring these records from a stronger agent improves the weaker agent’s task success rate and reduces its mean execution time and inference cost on successful trials. Execution time, inference cost, and reliability in coordinated deformable object manipulation remain key challenges. These results support general purpose agents as a viable approach to robot control through reasoning, programming, and physical interaction.

AI USE STATEMENT

During the preparation of this manuscript, the authors used generative AI tools to assist with drafting and language editing, including refining sentence structure and improving clarity and readability. The authors are responsible for reviewing and revising the resulting text to ensure that it accurately represents the research. The authors take full responsibility for the content of the manuscript, including the methodology, experimental results, interpretations, and conclusions.

REPRODUCIBILITY STATEMENT

Section 3 describes the agent formulation, robot interface, and runtime execution procedure. Section 4.1 summarizes the experimental setup, with robot hardware, control parameters, and agent configurations detailed in Appendix A. Appendix B describes the tasks, reference materials, success criteria, trial protocol, and metric definitions. The assembly parts are adapted from the AutoMate dataset (Tang et al., 2024), as described in Appendix B.1. Appendix C describes the supplementary coding agent comparison on pyramid construction and its usage and cost accounting. Appendix D documents the execution time analysis and repeated execution study, including experience checkpoints and timing boundaries. Appendix F presents visual references and representative execution sequences.

REFERENCES

- Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as i can, not as i say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- Anthropic. Claude Code overview. <https://code.claude.com/docs/en/overview>, a. Accessed September 8, 2026.
- Anthropic. Claude Code model configuration. <https://code.claude.com/docs/en/model-config>, b. Accessed September 8, 2026.
- Anthropic. Claude Opus 5 system card. <https://www-cdn.anthropic.com/b514064af1408018e64blad24e7d5e75850b4ffd/Claude%20Opus%205%20System%20Card.pdf>, 2026. Accessed September 8, 2026.
- Kevin Black, Noah Brown, Danny Driess, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- Anthony Brohan, Noah Brown, Justice Carbajal, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.
- Junting Chen, Yao Mu, Qiaojun Yu, Tianming Wei, Silang Wu, Zhecheng Yuan, Zhixuan Liang, Chao Yang, Kaipeng Zhang, Wenqi Shao, Yu Qiao, Huazhe Xu, Mingyu Ding, and Ping Luo. RoboScript: Code generation for free-form manipulation tasks across real and simulation. *arXiv preprint arXiv:2402.14623*, 2024.
- Kaiyuan Chen, Shuangyu Xie, Letian Fu, Justin Yu, William Pacini, Sandeep Bajamahal, Hudson Kim, Jaimyn Drake, Daehwa Kim, Haoru Xue, Jonathan Francis, Christian Juetter, Peter Schaldenbrand, Muhammet Yunus Seker, Ruwan Wickramarachchi, Uksang Yoo, Guanzhi Wang, Adithyavairavan Murali, Balakumar Sundaralingam, S. Shankar Sastry, Spencer Huang, Yuke Zhu, Linxi Fan, and Ken Goldberg. GaP: A graph-as-policy multi-agent self-learning harness for variational automation tasks. *arXiv preprint arXiv:2607.05369*, 2026.

- Cheng Chi, Zhenjia Xu, Siyuan Feng, Eric Cousineau, Yilun Du, Benjamin Burchfiel, Russ Tedrake, and Shuran Song. Diffusion policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*, 2023.
- Liane Galanti, Dhruv Shah, and Tri Dao. Addressing the orchestration gap in generalist robots via physical agency. *arXiv preprint arXiv:2607.21725*, 2026.
- Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annual Review of Control, Robotics, and Autonomous Systems*, 4:265–293, 2021. doi: 10.1146/annurev-control-091420-084139.
- Sirui Hong, Mingchen Zhuge, Jiaqi Chen, et al. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023.
- Wenlong Huang, Fei Xia, Ted Xiao, et al. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- I2RT. I2RT robot software and YAM platform. <https://github.com/i2rt-robotics/i2rt>. Accessed September 8, 2026.
- Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, et al. OpenVLA: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- Ruiying Li, Yunlang Zhou, Yu Yao Zhu, Kylin Chen, Jingyuan Wang, Sukai Wang, Kongtao Hu, Minhui Yu, Bowen Jiang, Zhan Su, Jiayao Ma, Xin He, Yongjian Shen, Yang Yang, Guanghui Ren, Maoqing Yao, Wenhao Wang, and Yao Mu. RoboClaw: An agentic framework for scalable long-horizon robotic tasks. *arXiv preprint arXiv:2603.11558*, 2026a.
- Shuang Li, Yihuai Gao, Dorsa Sadigh, and Shuran Song. Unified video action model. *arXiv preprint arXiv:2503.00200*, 2025.
- Zongxia Li, Xiyang Wu, Hongyang Du, Fuxiao Liu, Huy Nghiem, and Guangyao Shi. A survey of state of the art large vision language models: Alignment, benchmark, evaluations and challenges. *arXiv preprint arXiv:2501.02189v7*, 2026b. URL <https://arxiv.org/abs/2501.02189v7>. Revised August 22, 2026.
- Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, and Andy Zeng. Code as policies: Language model programs for embodied control. *arXiv preprint arXiv:2209.07753*, 2022.
- Runyu Lu, Yubo Wu, Ethan Kou, Letian Fu, Wenli Xiao, Ajay Mandlekar, Yinzhen Xu, Guanya Shi, Ken Goldberg, Ang Chen, Mosharaf Chowdhury, Yuke Zhu, Linxi Fan, and Guanzhi Wang. ASPIRE: Agentic /skills discovery for robotics. *arXiv preprint arXiv:2607.00272*, 2026.
- Yao Mu, Junting Chen, Qing-Long Zhang, Shoufa Chen, Qiaojun Yu, Chongjian Ge, Runjian Chen, Zhixuan Liang, Mengkang Hu, Chaofan Tao, Peize Sun, Haibao Yu, Chao Yang, Wenqi Shao, Wenhai Wang, Jifeng Dai, Yu Qiao, Mingyu Ding, and Ping Luo. RoboCodeX: Multimodal code generation for robotic behavior synthesis. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 36434–36454. PMLR, 2024.
- OpenAI. Codex CLI. <https://learn.chatgpt.com/docs/codex/cli>, a. Accessed September 8, 2026.
- OpenAI. GPT-6 Astra model. <https://developers.openai.com/api/docs/models/gpt-6-astra>, b. Accessed September 8, 2026.
- OpenAI. OpenAI model catalog. <https://developers.openai.com/api/docs/models/all>, c. Accessed September 8, 2026.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *arXiv preprint arXiv:2302.04761*, 2023.

- Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- Ishika Singh, Valts Blukis, Arsalan Mousavian, Ankit Goyal, Danfei Xu, Jonathan Tremblay, Dieter Fox, Jesse Thomason, and Animesh Garg. ProgPrompt: Generating situated robot task plans using large language models. *arXiv preprint arXiv:2209.11302*, 2022.
- Bingjie Tang, Iretiayo Akinola, Jie Xu, Bowen Wen, Ankur Handa, Karl Van Wyk, Dieter Fox, Gaurav S. Sukhatme, Fabio Ramos, and Yashraj Narang. AutoMate: Specialist and generalist assembly policies over diverse geometries. *arXiv preprint arXiv:2407.08028*, 2024.
- Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang H. Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Yanjun Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. *arXiv preprint arXiv:2407.16741*, 2024.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, et al. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.
- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *arXiv preprint arXiv:2404.07972*, 2024.
- John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.
- Seonghyeon Ye, Yunhao Ge, Kaiyuan Zheng, Shenyuan Gao, Sihyun Yu, George Kurian, Suneel Indupuru, You Liang Tan, Chuning Zhu, Jiannan Xiang, et al. World action models are zero-shot policies. *arXiv preprint arXiv:2602.15922*, 2026.
- Kevin Zakka. Mink: Python inverse kinematics based on MuJoCo. <https://github.com/kevinzakka/mink>, 2026.
- Yixian Zhang, Huanming Zhang, Feng Gao, Xiao Li, Zhihao Liu, Yi Nie, Chunyang Zhu, Jiaying Qiu, Yuchen Yan, Jiyuan Liu, Wenhao Tang, Jiayi Rao, Zhengru Fang, Changxu Wei, Yu Wang, Wenbo Ding, and Chao Yu. Harness VLA: Steering frozen VLAs into reliable manipulation primitives via memory-guided agents. *arXiv preprint arXiv:2607.08448*, 2026.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

Contents of Appendix

A	Implementation Details	13
A.1	Robot Platform and Control Interface	13
A.2	Agent Configuration and Execution Conditions	14
A.3	Model and Reasoning Effort Configurations	14
B	Tasks and Evaluation Protocol	14
B.1	Task Definitions and Success Criteria	14
B.2	Trial Protocol	15
B.3	Metric Definitions and Resource Accounting	15
C	Supplementary Coding Agent Comparison on Pyramid Construction	16
D	Efficiency and Experience Reuse Analysis	17
D.1	Task Execution Time Analysis	17
D.2	Repeated Execution with Persistent Experience	18
D.3	Transferring Frozen Experience to Terra	18
E	Runtime Program Analysis	19
F	Qualitative Task Examples	20
F.1	Four Pair Assembly	20
F.2	Block Construction	22
F.3	Dice Flipping	23
F.4	Targeted Throwing	24
F.5	Bimanual Towel Folding	26
G	Execution Across Arm and End Effector Embodiments	30
H	Error and Recovery Study	32
H.1	Six Block Tower Collapse During Withdrawal	32
I	Broader Impact	33

A IMPLEMENTATION DETAILS

A.1 ROBOT PLATFORM AND CONTROL INTERFACE

Figure 4 shows the platform described in Section 4.1.1.

Robot platform. The YAM arm uses a linear_4310 parallel gripper with a nominal maximum inner jaw opening of 95.5 mm. The i2rt Python library supplies robot models, motor communication, and gravity compensation over a 1 Mbit/s CAN connection. Bimanual Cartesian poses share a coordinate frame through a calibrated rigid transform.

Observations and geometric grounding. Captures from the cameras specified in Section 4.1.1 include calibration, timestamps, and robot state. Wrist depth maps image pixels into the arm base frame. Overhead pixels are located by intersecting camera rays with a plane at an estimated height. The agent can request additional views to refine these estimates. Camera stream rates are independent of the agent’s decision rate.

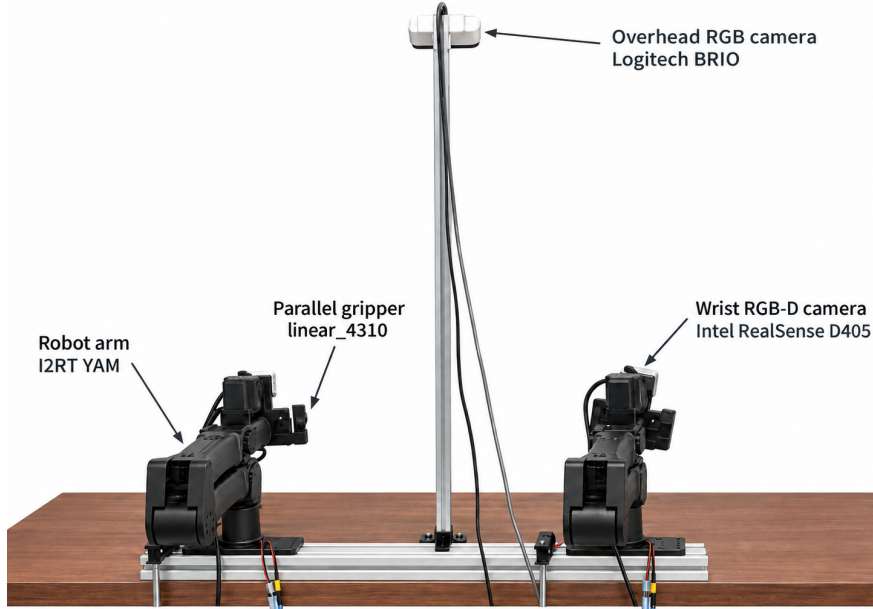


Figure 4: **Robot platform and camera placement.** The setup contains two I2RT YAM arms mounted on a shared tabletop rail. The annotations identify an arm, a linear_4310 parallel gripper, a wrist mounted Intel RealSense D405 RGB and depth camera, and the fixed overhead Logitech BRIO RGB camera. The two arms support bimanual towel folding.

Action interface. The robot interface provides seven commands. `state` returns joint positions, the end effector pose, gripper opening fraction and width, and robot health information. `status` reports the operating status of the robot interface, request budget usage, and coordinate frame information. `help` lists the available commands and their arguments. `frames` captures images from the requested cameras, optionally saves wrist depth, and returns paths to the saved observations and calibration data.

`move_ee` requests a target end effector position and quaternion orientation in the shared coordinate frame and returns the achieved pose and target error, or failure information. `move_joints` requests target joint angles in radians and returns the resulting joint positions and end effector pose. `grripper` requests opening, closing, or a normalized opening fraction between 0 and 1 and returns the resulting opening fraction and width. The agent computes geometry from saved images, depth, and calibration in local programs. `Cartesian motion` invokes the underlying inverse kinematics and trajectory

execution software. Ordinary motion uses joint velocity and acceleration limits of $20^\circ/\text{s}$ and $40^\circ/\text{s}^2$, a Cartesian translation limit of 0.03 m/s , and an orientation limit of $10^\circ/\text{s}$. The gripper target changes at 0.25 of its normalized travel per second. The session server enforces a radial target envelope of 0.12 to 0.65 m, a grasp point height envelope of -0.050 to 0.60 m , and a maximum Cartesian step of 0.25 m . For targeted throwing, the recorded case uses a task specific runtime with timed motion programs to coordinate the swing and gripper release. Each program specifies joint offsets at time knots and scheduled gripper targets. The executor interpolates commands on a 50 Hz grid and records measured joint position, velocity, effort, and dispatch timing. Within these programs, joint 4 uses velocity and acceleration limits of $180^\circ/\text{s}$ and $360^\circ/\text{s}^2$.

A.2 AGENT CONFIGURATION AND EXECUTION CONDITIONS

Trials use the inputs and workspace defined in Section 3.1. Standard trials start with empty task experience. Experience reuse conditions supply the saved files specified in Section 4.3. Each trial records model identifiers, agent software versions, host specifications, budgets, and inference settings.

The execution efficiency instructions in Section 3.3 are held fixed within each condition. Agent traces record waiting intervals, intermediate status checks, and operations grouped within each tool call.

A.3 MODEL AND REASONING EFFORT CONFIGURATIONS

The configurations are listed in Table 2. The Astra effort sweep holds the model version, agent environment, task inputs, and robot interface fixed. Comparisons across Codex and Claude Code evaluate complete agent systems under shared scenes, references, and budgets. Claude Code model and effort settings use its native controls (Anthropic, b). Claude Opus 5 uses identifier `claude-opus-5`. Each trial records the requested and resolved model identifiers and effective effort setting.

B TASKS AND EVALUATION PROTOCOL

B.1 TASK DEFINITIONS AND SUCCESS CRITERIA

Table 3 defines the tasks, target capabilities, instruction types, and success criteria used in the main evaluation. Controller arrival tolerances are reported separately from task success tolerances.

Assembly from video. The AutoMate parts (Tang et al., 2024) form a ridged sleeve and broad round base, a hexagonal nut and short post, a smooth sleeve and stepped cylinder, and a long pin and hollow tube, in demonstration order. All methods receive the same human video, extracted frames, and timestamped textual guide. Figure 5 shows an execution.

Assembly part dimensions. Table 4 reports the nominal printing dimensions and mating clearances of the AutoMate meshes, scaled with their aspect ratios preserved.¹ Successful assembly with submillimeter nominal radial clearances illustrates precise alignment (Figure 5). Physical insertion tolerance also depends on printing accuracy and angular alignment.

Block construction from images. Initial cube poses vary across trials while the goal photograph remains fixed within each configuration. Block sets and placement tolerances are fixed before evaluation. The three configurations appear in Figure 6.

Action diversity. Dice flipping uses distinct initial and target faces balanced across predefined conditions. Grasp selection, rotation, and regrasping are annotated from video. For throwing, object and target geometry, initial arrangement, and permitted swing conditions are fixed before evaluation. Landing error is measured relative to the target center. Execution records and video establish joint motion, gripper opening, and object separation.

¹The plug and socket meshes are `asset_plug.obj` and `asset_socket.obj` in each official AutoMate asset directory.

Table 3: **Task definitions and success criteria.** Capability identifies the primary manipulation challenge, and Instruction type identifies how the task goal or procedure is communicated. The final column specifies the required behavior and physical outcome. Success is assessed from the final state and execution video using criteria fixed before evaluation. Required assembly, support, orientation, and fold relationships must remain stable after gripper release and arm withdrawal.

Task	Capability	Instruction type	Task requirements and success criteria
Four pair assembly	Long horizon reasoning	Video	Assemble eight parts into four pairs in the demonstrated order, inferring mating relationships and insertion motions from the video and current scene. Success requires all four pairs to remain assembled, preserving completed pairs throughout subsequent operations.
Block construction	Long horizon reasoning	Image	Arrange six cubes into a pyramid, two towers, or a six block tower matching the goal image. Infer object identities, orientations, and support relations, and choose a placement order that maintains intermediate stability. Success requires the complete target structure.
Dice flipping	Action diversity	Language	Reorient six randomly placed dice to the requested upward face through grasp selection and rotation. Success requires all six dice to show the requested number after placement.
Targeted throwing	Action diversity	Language	Coordinate a circular joint swing with gripper release to throw an object toward the specified target. Success requires the object to separate from the moving hand, travel through the air, and come to rest within the target region.
Towel folding	Object diversity	Video	Use both arms to reproduce the demonstrated folds. Fold the near long edge onto the far edge, then fold both short ends inward, either right then left (Sequential) or beginning together (Simultaneous). Success requires both inward folds with reasonably aligned edges. Small wrinkles and creases are permitted, and the final table position is unrestricted.

Bimanual towel folding. Each configuration provides an overhead demonstration video and images of the initial and final towel states. Trials begin with the towel approximately matching the demonstrated initial arrangement. The agent may stage placements to maintain arm clearance. Final folds are assessed from overhead images after release and withdrawal. Figures 9 and 10 show the two configurations.

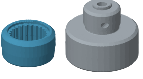
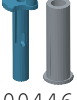
B.2 TRIAL PROTOCOL

Trials follow Section 4.1.2. The records distinguish physical failure, budget exhaustion, human intervention, and infrastructure interruption under a predefined inclusion policy. Geometric queries and local computation are logged separately from budgeted observation and action requests.

B.3 METRIC DEFINITIONS AND RESOURCE ACCOUNTING

Success and time. Success rate is $100s/n$, where s is the number of completely successful trials among n evaluated trials. The task criteria are specified in Table 3. Completion time, token usage, and cost use the same successful trials, with sample counts and summary statistics reported in Tables 1 and 2. A configuration with no successes has undefined successful trial resource means.

Table 4: **Printing dimensions and mating clearances.** Assembly identifies each AutoMate part pair in demonstration order, with plug and socket meshes shown in blue and gray, respectively. Plug overall size and Socket overall size give the respective mesh dimensions as $x \times y \times z$ in the original mesh frame. Hole D_h is the effective bore diameter, measured by the largest cylinder that fits through the bore, including its narrowest ridges. Shaft D_s is the effective shaft diameter, measured by the smallest cylinder enclosing the mating shaft segment. The diametral clearance is $\Delta = D_h - D_s$, and the conservative radial clearance $c = \Delta/2$ gives the available transverse offset for parallel mating axes. All dimensions and clearances are in millimeters. The hole is in the plug mesh for the first three pairs and in the socket mesh for 00446. Clearances are computed before rounding.

Assembly	Plug overall size	Socket overall size	Hole D_h	Shaft D_s	Δ	c
 00081	$52.95 \times 52.98 \times 27.53$	$71.59 \times 71.65 \times 58.26$	37.41	35.86	1.56	0.78
 00581	$55.58 \times 48.32 \times 19.46$	$45.52 \times 39.42 \times 78.83$	28.18	25.80	2.38	1.19
 00007	$51.08 \times 50.94 \times 39.60$	$42.42 \times 42.30 \times 81.44$	42.86	37.20	5.66	2.83
 00446	$39.25 \times 39.16 \times 105.04$	$31.32 \times 31.32 \times 90.81$	18.01	16.54	1.47	0.73

Completion time follows the task delivery to final physical verification boundary in Section 4.1.2. It includes program construction, observation, computation, motion, and recovery. The broader timing analysis in Appendix D.1 also includes unsuccessful trials and terminal reporting.

Interaction accounting. The index k in Equation 1 counts agent tool decisions. The interaction budget counts individual observation and action requests, including those grouped within a single tool call.

Token usage. Total tokens sum input and output usage across all model requests in a trial, including retries. Repeated context counts on each request, with cached input counted once within that request’s input total. Provider records are normalized so that reasoning contributes once to output. Table entries report mean total tokens per successful trial in millions.

C SUPPLEMENTARY CODING AGENT COMPARISON ON PYRAMID CONSTRUCTION

Table 5 extends the agent comparison to pyramid construction using GPT-5.6 Sol and GPT-6 Astra within Codex at high thinking effort. The task follows Table 3. Sol’s unsuccessful sessions comprise two supervisor interventions with incomplete structures and one robot bridge disconnection before physical execution.

Usage and pricing. Per request usage is checked against cumulative counters using the token convention in Appendix B.3. API equivalent costs use Standard prices retrieved on September 9, 2026. Executions use Pro subscriptions. Astra is configured for Fast service, with costs normalized to Standard prices for comparison.

Table 5: **Supplementary coding agent comparison on pyramid construction.** Both Codex models use high thinking effort, extending the comparison in Table 2. Successes count completed pyramids recorded in the session reports. Interrupted sessions count as unsuccessful. Tokens and cost are means over successful sessions for each model. Total cost covers all ten sessions for each model. Cost uses Standard API prices for each model.

Model	Successes	Tokens (million)	Cost (USD)	Total (USD)
GPT-5.6 Sol	7/10	18.86	9.22	87.25
GPT-6 Astra	10/10	9.24	11.69	116.88

Table 6: **Time allocation across manipulation tasks.** Values are mean minutes over all N evaluated trials, including failures. Total includes final reporting. Throwing also includes trajectory analysis and verification. The components sum to Total before rounding.

Task	N	Total	Visual loop	Remaining response time	Local tools	Action service	Check	Report
Four pair assembly	10	48.5	25.5	9.9	0.3	11.6	0.5	0.8
Block construction								
<i>Pyramid</i>	10	22.3	8.7	3.4	0.1	8.8	0.6	0.7
<i>Two towers</i>	10	21.4	8.5	4.7	0.1	7.1	0.2	0.9
<i>Six block tower</i>	10	29.8	14.1	5.0	0.1	9.3	0.5	0.8
Dice flipping	10	38.6	16.2	7.7	0.2	12.9	0.8	0.8
Targeted throwing								
<i>Potato</i>	2	23.7	7.2	12.7	0.7	1.9	0.2	0.9
Towel folding								
<i>Sequential</i>	5	52.7	25.4	10.0	0.5	14.7	0.3	1.9
<i>Simultaneous</i>	5	23.2	12.2	1.6	0.1	7.1	0.3	2.0

Interpretation. Sol uses more tokens per successful session and has lower estimated inference cost than Astra.

D EFFICIENCY AND EXPERIENCE REUSE ANALYSIS

D.1 TASK EXECUTION TIME ANALYSIS

Table 6 reports mean times for all trials represented in Table 1, including failures. Timing runs from task delivery to agent completion or timeout and includes final reporting. This differs from the main table, which summarizes successful trials through physical verification.

Time categories. The visual loop combines image capture, geometric queries, image processing, and the model response following an image. That response can include planning and code generation. Remaining response time covers model responses and associated delays outside the other categories. Local tools covers file access and local computation. Action service covers robot request processing, including motion planning, movement, settling, and rejected requests. Check covers final verification after the last action, and Report covers writing the final result through agent completion. Overlapping activities, including simultaneous arm movements, count once. These categories describe observable activity, with visual reasoning and programming measured together.

Observed costs. Table 6 shows that observing the scene and deciding how to act account for substantial execution time. The balance varies by task. Throwing spends more time in other model responses, whereas assembly and towel folding devote a large share to the visual loop. Simultaneous folding has shorter total time than sequential folding, with less time spent in both the visual loop and robot execution. Task strategy and trial outcomes also differ, so these timings alone do not isolate the benefit of parallel arm motion.

Table 7: **Execution breakdown over repeated tasks.** Times are seconds. The two Task execution columns and Experience writing sum to Total runtime, up to rounding. Retries counts failed physical stage attempts. Calls lists motion and gripper requests followed by image capture requests from task start through experience saving.

Cycle	Total runtime	Task execution		Experience writing	Counts	
		Reasoning and programming	Tool execution		Retries	Calls
Ring disassembly and reassembly						
1	989.7	702.9	167.4	119.4	1	37 / 37
2	522.9	282.1	126.7	114.1	0	32 / 33
3	489.9	245.0	127.3	117.6	0	32 / 33
4	484.4	247.7	124.6	112.2	0	32 / 33
5	461.7	268.3	125.0	68.5	0	32 / 33
Two pair assembly						
1	621.4	351.9	160.5	108.9	0	25 / 27
2	568.1	281.7	186.3	100.1	0	23 / 25
3	507.5	237.1	173.8	96.6	0	23 / 25
4	491.8	221.0	174.1	96.6	0	23 / 25
5	462.4	186.6	175.5	100.3	0	23 / 25

D.2 REPEATED EXECUTION WITH PERSISTENT EXPERIENCE

Task protocols. Each task in Section 4.3.1 is repeated five times using GPT-6 Astra at high thinking effort. The first execution starts with empty experience. Each later execution uses a fresh agent context with saved measurements, programs, and lessons from earlier attempts. The agent updates object positions from current images.

For the ring task, each cycle starts assembled, removes the ring onto the table, and reassembles it on the base. All cycles succeed after release and withdrawal, with video and action logs documenting execution. The two pair task assembles a hexagonal ring onto a narrow post and a circular collar onto a stepped cylinder. The agent reports stable assemblies in every trial based on recorded images.

Time measurements. Task time runs from the first task start marker to successful assembly verification, including the return to the observation posture for two pair assembly. Experience writing covers the subsequent saving of updated experience. Within task time, Tool execution measures recorded time spent running programs, processing images, and operating the robot. Simultaneous activities count once. Time when tools run while the model responds is assigned to Tool execution. The remaining task time estimates Reasoning and programming, including planning, code generation, and delays in receiving model responses or starting tools. This definition makes the columns additive while treating model activity as a combined estimate.

What experience changes. Table 7 shows that the main time savings occur in reasoning and programming. For two pair assembly, this time decreases while tool execution takes slightly longer. For the ring task, the largest improvement follows the first cycle, when the agent establishes an alignment procedure and recovers from a failed insertion.

The logs show how later agents use the saved experience. They call existing measurement and motion programs, update object coordinates in earlier scripts, and reuse corrected grasp and insertion procedures. These observations link shorter response times with reuse of programs and task knowledge. The trials form an ordered sequence with changing object positions, so a comparison that starts every trial with empty experience is needed to isolate the effect of experience reuse.

D.3 TRANSFERRING FROZEN EXPERIENCE TO TERRA

The transfer study in Section 4.3.2 gives every Terra trial the same frozen Astra experience files. Operator review confirms successful assembly in four trials. In the unsuccessful trial, the circular pair is completed, but the hexagonal collar remains separate and the narrow post tips during recovery.

Table 8: **Programs constructed during execution.** Primary purposes of the 37 retained Python working files. Image crops, demonstration montages, and some object detection routines also appear as inline code and are outside this file inventory.

Primary purpose	Files	Observed computations and operations
Perception and geometry	29	Depth measurement, color region selection, ray intersection, triangulation, surface fitting, and grasp offset estimation.
Robot call composition	7	Pose and gripper commands followed by capture, concurrent calls to different arms, compact responses, and full response logging.
Visual annotation	1	Projection of spatial reference points into camera images for inspection.

The empty experience comparison uses the Terra trials in Table 2. Time and resource accounting follow that table. Completion is marked by the final result update after motion execution, or the recorded task completion time when no subsequent update exists. Costs cover Terra inference at Standard rates. The resource means compare one successful trial without experience with four using experience, so the observed reductions rest on small and unequal samples.

E RUNTIME PROGRAM ANALYSIS

We analyze the programs constructed during 65 robot task executions to characterize the runtime programming described in Section 3.3.

Program inventory. The session workspaces retain 37 Python working files across 18 sessions. Agent event logs identify their creation or renaming during execution. Table 8 groups these files by their primary purpose. We count each retained working path once within its session. The inventory separately identifies 63 Python file copies supplied through inherited knowledge and seven copies exported for subsequent reuse. These copies are excluded from the 37 working files.

The logs also contain 135 shell command events with inline Python across 48 sessions, including all 18 sessions with saved working files. These events contain image processing and geometric computation as well as dependency checks, report writing, and file verification. An event can combine several operations or create a saved program, so event and file counts are reported separately. The inventory reflects the available interfaces, which provide either geometric queries or images, depth, and calibration for local computation.

Surface fitting and grasp offset compensation. In a four pair assembly execution, the agent creates `fit_head.py` to estimate the pose of a held pin from a wrist depth image. It samples triples of points to identify a planar surface, then uses singular value decomposition to estimate its normal. The following excerpt refines the plane estimate from the selected surface points.

```
ctr = best.mean(0)
_, _, v = np.linalg.svd(best - ctr)
normal = v[-1]
normal *= np.sign(normal[2])
```

The agent extends the program to intersect an overhead pixel ray with this plane, estimate the pin tip using an assumed pin length, and express the tip offset in the gripper frame. A further extension computes a corrected target orientation and position. The resulting horizontal target is approximately (0.51325, -0.15820) m. The next motion request uses (0.5133, -0.1582) m and the computed quaternion, establishing a direct link between the program output and physical action selection. That request returns a target arrival error. The agent then adjusts the requested position and continues the insertion sequence. This trace illustrates successive program extensions and the use of their outputs within a feedback loop.

Triangulation from two camera views. In another four pair assembly execution, the agent creates `triangulate.py`. It converts selected wrist and overhead pixels into calibrated rays in the robot

base frame. Given ray origins t_1 and t_2 and unit directions d_1 and d_2 , the program computes the closest points on the two rays using the following operations.

```
ab = np.linalg.lstsq(
    np.stack([d1, -d2], 1), t2 - t1, rcond=None
)[0]
p1 = t1 + ab[0] * d1
p2 = t2 + ab[1] * d2
```

It returns their midpoint as the position estimate and their separation as a consistency measure. The agent subsequently calls the same program on additional targets. Pixel correspondences are supplied by the agent, and the ray separation measures agreement between those inputs under the saved calibration. Position accuracy also depends on correspondence and calibration error.

Visual processing and execution helpers. Other executions contain inline programs for color segmentation, circle detection with multiple thresholds, and demonstration frame montages. During bimanual towel folding, the agent creates `step.py` to read both arm states, dispatch one operation per arm concurrently, and capture observations after the operations. It also retains full responses in a local log while printing compact results. This helper combines the interface operations described in Section 3.2. The cases document program construction and use, while controlled comparisons would be needed to measure their contribution to task performance.

F QUALITATIVE TASK EXAMPLES

F.1 FOUR PAIR ASSEMBLY

Figure 5 shows recovery during the four pair assembly task. An unsuccessful insertion displaces the nut onto the table. The agent adjusts its grasp position and gripper orientation, seats the nut, and continues with the remaining assemblies while preserving earlier progress. The final frame shows all four pairs assembled after release and withdrawal.



Figure 5: **Four pair assembly from a human demonstration video.** The upper panel shows four demonstrated assemblies. The lower panel shows the robot assembling eight parts into four pairs, illustrating video instruction following and long horizon reasoning. Dark red bars mark failed nut insertion and displacement. Yellow bars mark grasp retries. Read each panel row by row. Timestamps give elapsed minutes and seconds in each video at unequal intervals.

F.2 BLOCK CONSTRUCTION

Figure 6 shows how placement order follows the target support structure. The pyramid is built tier by tier. The two towers are built together, with both bases placed before the middle and top cubes. The six block tower grows above a retained base, requiring alignment to be maintained over successive placements. All three structures remain standing after release and withdrawal.

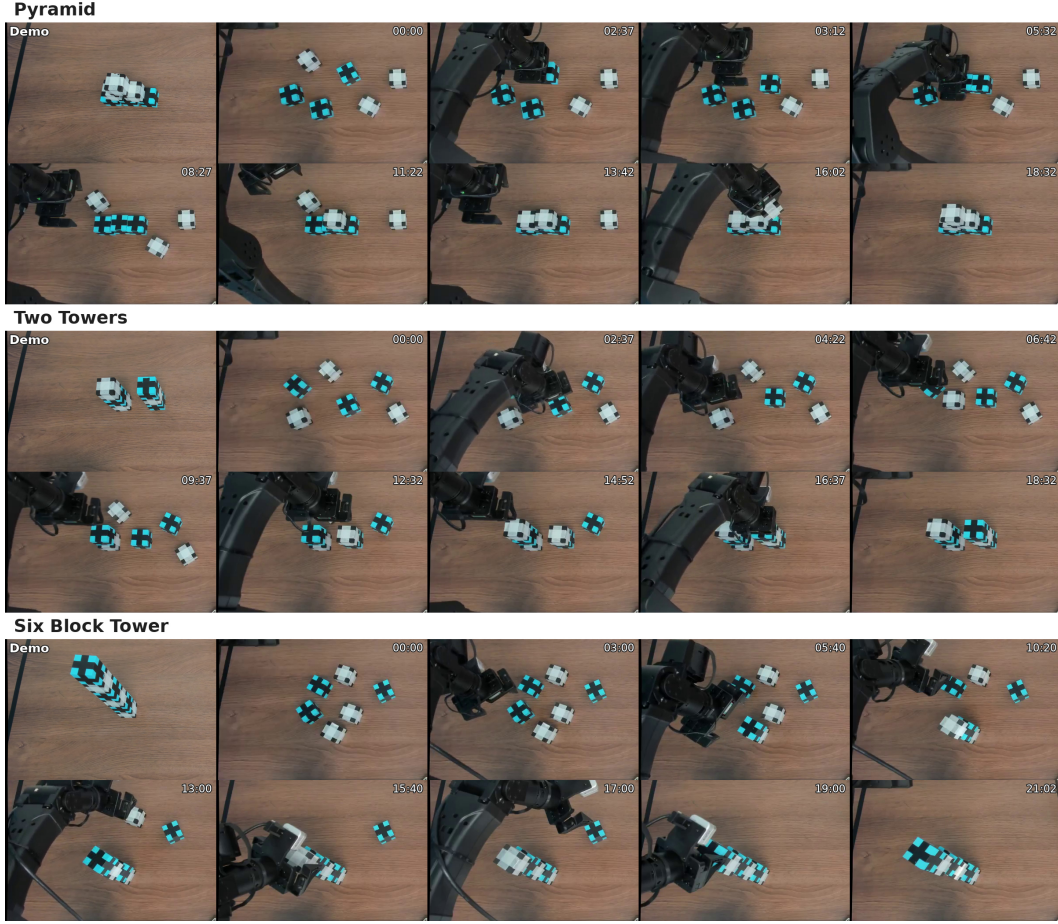


Figure 6: **Block construction from static goal images.** From top to bottom are the pyramid, two towers, and six block tower. Each occupies two rows of five images, beginning with the goal image labeled Demo and followed by nine execution frames read row by row. Timestamps give elapsed minutes and seconds in each recording at unequal intervals.

F.3 DICE FLIPPING

Figure 7 shows grasp adjustments that accommodate the arm’s reachable poses. The agent combines a 60° tilt with a low release so that a die settles onto the intended face. Further observations guide regrasping and verify orientation. The final frame shows all six target faces upward after withdrawal.

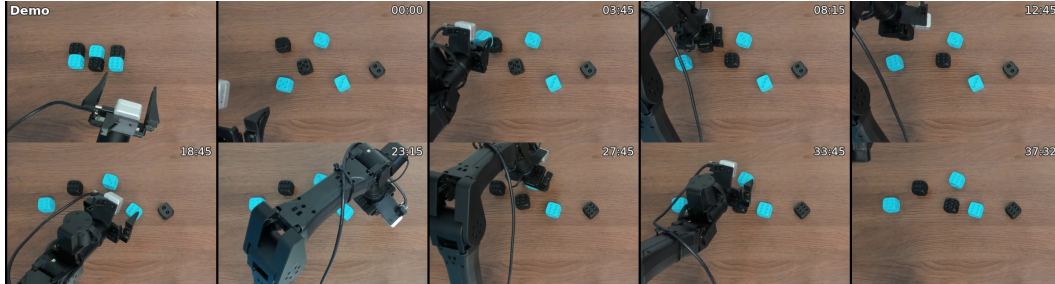


Figure 7: **Dice flipping across six dice.** The goal photograph in the cell labeled Demo specifies six pips upward on each of three black and three light blue dice. Nine overhead execution frames show the initial faces, successive manipulations, and final state with all six target faces upward. The task requires selecting grasps and rotations that produce the requested orientation after release. Read the execution frames row by row. Timestamps show elapsed minutes and seconds in the source video.

F.4 TARGETED THROWING

In Figure 8, the agent performs two motion probes while retaining the object, then adjusts the swing and gripper opening schedule. Joint 4 drives the swing while the other joint targets remain fixed. The views show separation from the gripper and the object resting inside the bowl. Occlusion obscures part of the rising trajectory and its apex, so the full throwing assessment also uses the execution records and video.



Figure 8: **Targeted throwing from a language instruction.** The task requires a joint driven circular swing and timed gripper release toward a bowl, illustrating the dynamic release component of action diversity. The overhead sequence shows the initial arrangement, grasp, swing, and final object position inside the bowl. The wrist sequence shows retention and separation at the gripper. Occlusion limits the visible flight trajectory. Read each grid row by row. Timestamps show elapsed minutes, seconds, and milliseconds on a common clock, with denser sampling around release and landing.

F.5 BIMANUAL TOWEL FOLDING

Sequential folding. In Figure 9, a corner slips during the initial paired carry. The agent recovers it and corrects a turned back edge before bringing the doubled towel closer to the arm bases. One arm then holds the towel while the other folds an end inward. Further grasps flatten a raised corner.

The final towel preserves the demonstrated fold order after release and withdrawal. A buckle and edge offsets remain, and the execution report judges the result a partial visual match. The sequence illustrates successful recovery of the fold arrangement with residual alignment errors.

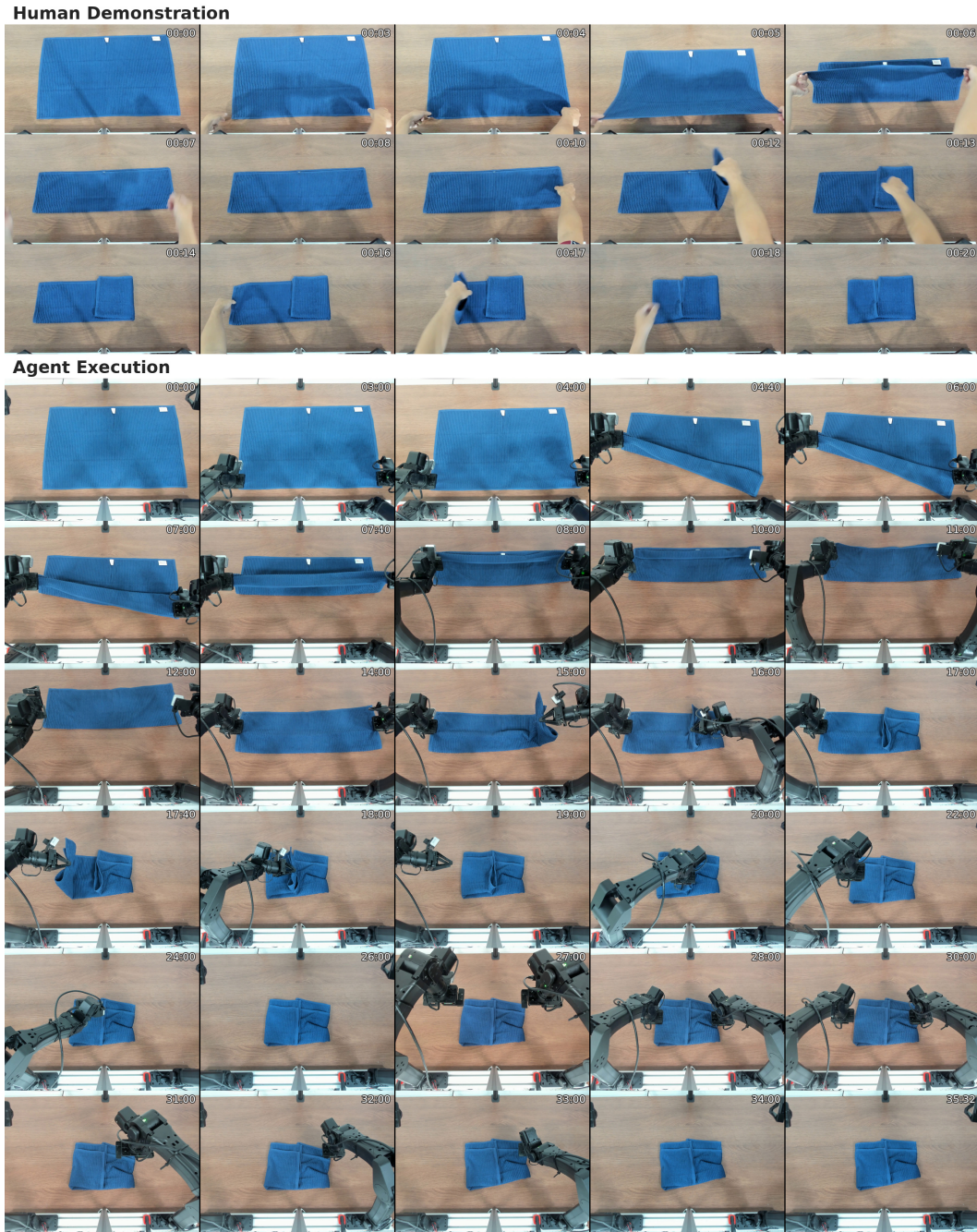


Figure 9: **Sequential bimanual towel folding from a human demonstration video.** The upper panel contains 15 frames showing the long edge fold followed by the right and left ends folding inward. The lower panel contains 30 frames showing coordinated grasping, recovery after a corner slips, the three folds, and subsequent adjustments. The final towel preserves the demonstrated fold arrangement with a remaining buckle and edge offsets. Read each panel row by row. Timestamps show elapsed minutes and seconds in the respective source videos, with unequal sampling intervals.

Simultaneous folding. In Figure 10, one arm holds its corner while the other recovers a slipped edge. After the long edge fold, both arms carry the short ends inward, then place the flaps in sequence to maintain clearance. A further correction anchors an outer edge while drawing a raised inner hem toward the center.

Both inward folds are completed after release and withdrawal, with residual skew, a curled edge, and a gap near the central seam. The trial is counted as successful under the criteria in Table 3. The coordinated inward transport and staged placement show how the agent adapts the demonstration to the two arms' workspace.

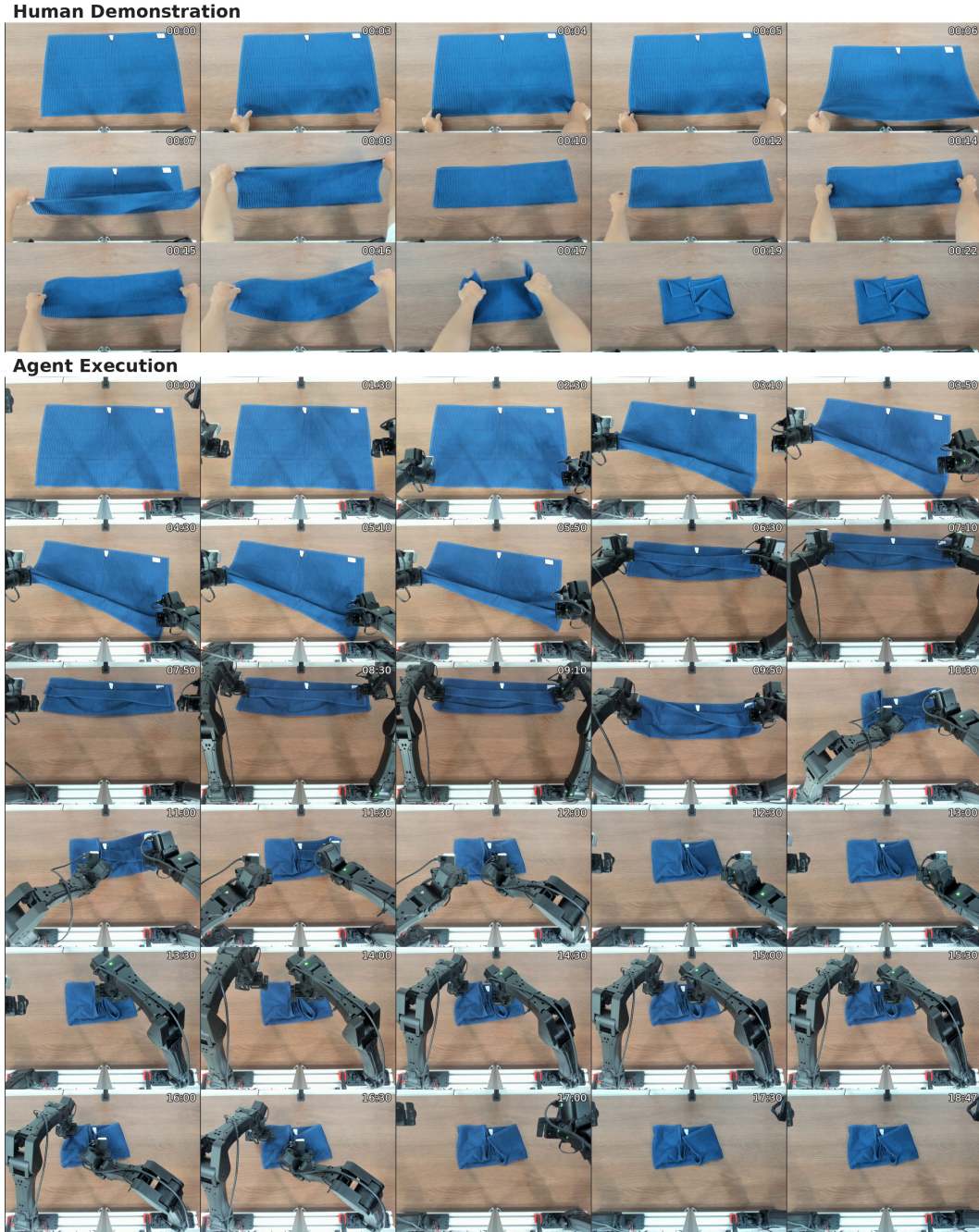


Figure 10: **Simultaneous bimanual towel folding from a human demonstration video.** The upper panel shows the long edge fold followed by both short ends folding inward together. The lower panel shows the robot execution, including recovery of a slipped corner, the long edge fold, inward transport of both ends, staged placement of the flaps, and alignment correction. The final state contains both inward folds with residual skew and edge offsets, with both arms withdrawn. Read each panel row by row. Timestamps show elapsed minutes and seconds in the respective source videos, with unequal sampling intervals.

G EXECUTION ACROSS ARM AND END EFFECTOR EMBODIMENTS

The bottle experiment in Figure 11 uses a seven joint P7 arm and a RealHand L6 dexterous hand. The hand has five fingers controlled through thumb flexion, thumb abduction, and one channel for each remaining finger. A fixed overhead RealSense D435i and a wrist mounted RealSense D435 provide color images at 640×480 pixels and 15 Hz. This setup tests the observation and action loop with different arm kinematics and finger control.

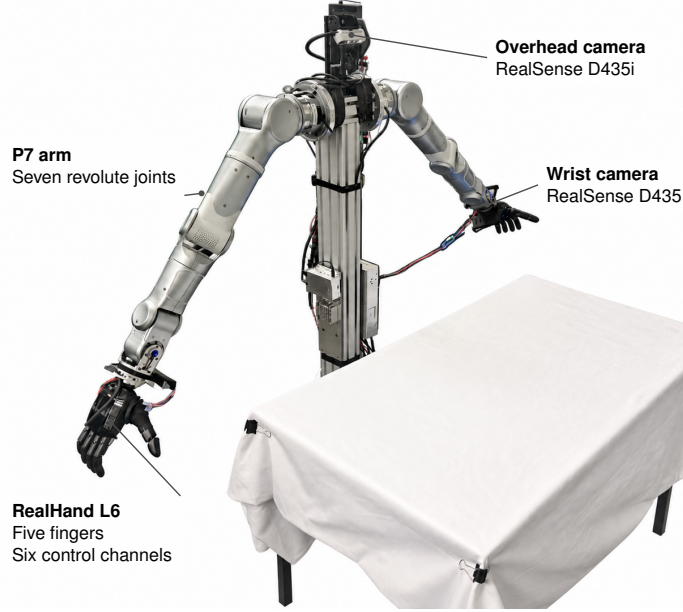


Figure 11: **P7 and L6 platform for the cross embodiment experiment.** The annotations identify the P7 arm, RealHand L6 dexterous hand, overhead RealSense D435i, and wrist mounted RealSense D435.

Figure 12 shows the grasp and lift sequence. From a stationary tabletop preparation pose, the agent reorients the wrist, opens the thumb, and approaches the bottle through successive position adjustments. It then closes the fingers in stages and commands a 2 cm upward wrist motion to raise the bottle. The bottle slips during subsequent holding. The demonstrated grasp and lift provides qualitative evidence that AGP can coordinate a different arm and dexterous hand through their available interfaces.

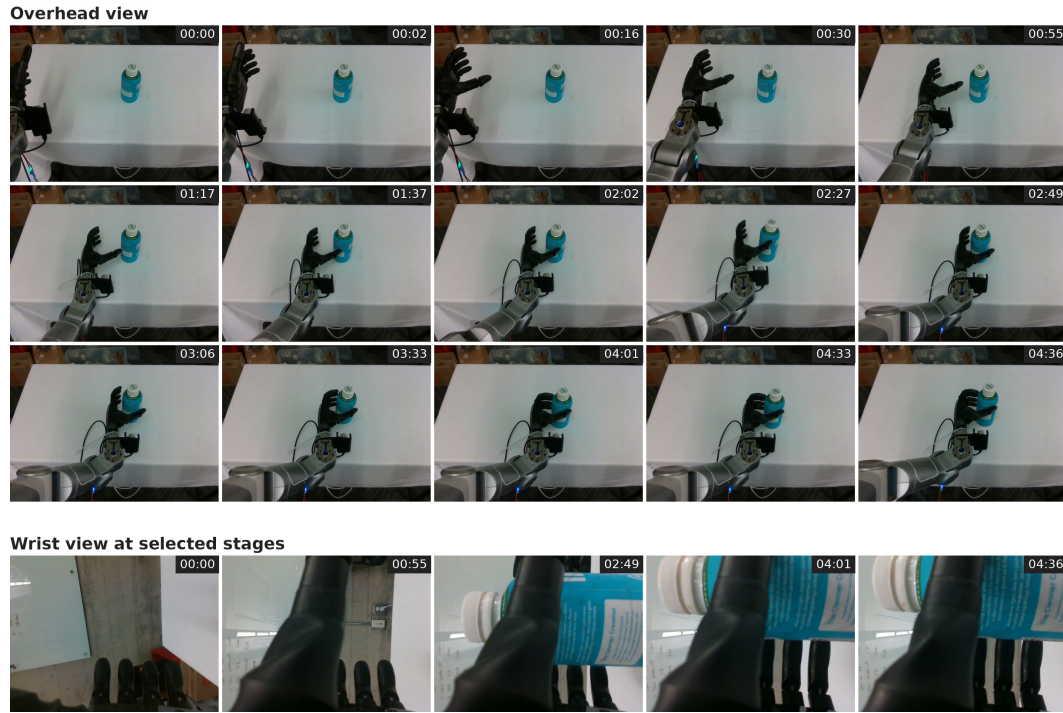


Figure 12: **Bottle grasp and lift with the P7 arm and L6 hand.** The upper panel contains 15 overhead frames read row by row, showing wrist reorientation, approach, finger closure, and lifting. The lower panel shows five corresponding stages from the wrist camera. Timestamps measure elapsed minutes and seconds from the first grasp action after stationary preparation. Frames are sampled at unequal intervals, with the last frame at 04:36. The two views are approximately aligned using their recorded host timestamps.

H ERROR AND RECOVERY STUDY

H.1 SIX BLOCK TOWER COLLAPSE DURING WITHDRAWAL

After the fourth cube turns out of alignment during placement, the agent regrasps and corrects it. To overcome limited reach at the sixth level, it assembles the top two cubes on the table and transfers them together by grasping the lower cube. All six cubes stand after release, but the tower collapses during the subsequent arm withdrawal. The observations establish when the collapse occurs, while the exact contact or instability that causes it remains unresolved.

The agent then localizes the scattered cubes to assess whether it can rebuild. Both overhead geometry and wrist depth place one grey cube near $x = 0.70$ m in the robot base frame, beyond the interface's 0.65 m radial grasp limit. It ends the attempt as unsuccessful despite having time and command budget remaining.

This case distinguishes a recoverable alignment error from a collapse that moves a required cube beyond reach. It also shows why stability during withdrawal is part of the success criterion.

I BROADER IMPACT

AGP may reduce the task specific engineering required to prototype manipulation from video, image, and language instructions. Saved programs and execution records could also support research on reusable robot skills.

A potential application is collecting robot trajectories for policy learning through repeated AGP execution. Synchronized visual observations, actions, and measured robot states could provide demonstrations for training vision language action policies, while failed attempts and recovery segments could support research on robustness. Its practical value would depend on trajectory quality, collection costs, and the performance of policies trained on these data.

Applying agents to physical systems introduces risks from inaccurate perception, faulty programs, and inappropriate motion requests, which can cause equipment damage or injury. Our implementation separates agent reasoning from motion execution and applies workspace and motion limits through the robot runtime. Deployment in settings involving people or unfamiliar environments requires additional safety evaluation, appropriate supervision, and independent protective mechanisms. Inference costs and execution latency also affect the practicality and accessibility of this approach.